

UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

ΕΦΑΡΜΟΓΗ ΔΗΜΟΠΡΑΣΕΙΩΝ

Student name: Πάυλος Μανίκας - Σταύρος Πανταζής
sdi: sdi2200092 - sdi2200131

Course: Τεχνολογίες εφαρμογών διαδικτύου
Semester: Αιαρινό εξάμηνο 2025

Contents

1	Prologe	3
2	Execution Instructions	3
3	Backend Django	4
3.1	Εισαγωγή	4
3.2	app summary	4
4	Frontend React	7
4.1	Εισαγωγή	7
4.2	Pages summaray	7
4.3	Τεστ	9
5	Epiloge	9
5.1	Problems	9
5.2	Summary	9

1. Prologe

Η παρούσα εργασία υλοποιήθηκε στα πλαίσια του μαθήματος «Τεχνολογίες Εφαρμογών Διαδικτύου» και έχει ως στόχο την ανάπτυξη μιας πλήρους διαδικτυακής εφαρμογής ηλεκτρονικών δημοπρασιών. Ο στόχος αυτής της εργασίας είναι να αναπτυχθεί μία εφαρμογή ηλεκτρονικών δημοπρασιών.

Στην εφαρμογή υπάρχουν 4 ρόλοι:

1. Διαχειριστής Εφαρμογής

Διαχειρίζεται την εφαρμογή και την σωστή της λειτουργία.

2. Πωλητής

Μπορεί να θέσει ένα ή περισσότερα αντικείμενα σε δημοπρασία.

3. Προσφοροδότης (bidder)

Μπορεί να πλοήγηθαι / να αναζητήσει αντικείμενα που έχουν θέσει σε δημοπρασία άλλοι χρήστες προκειμένου να υποβάλλει προσφορές.

4. Επισκέπτης

Μπορεί μόνο να πλοήγηθαι / να αναζητήσει αντικείμενα που έχουν θέσει σε δημοπρασία άλλοι χρήστες προκειμένου να υποβάλλει προσφορές.

2. Execution Instructions

Για να εξασφαλίσουμε την ομαλή δημιουργία κώδικα και την αποφυγή πιθανών λαθών λόγω της χρήσης μηχανισμάτων διαφορετικού λογισμικού για την εγγραφή κώδικα αποφασίσαμε να χρησιμοποιήσουμε την τεχνολογία **docker** και **docker-compose**.

Για να δημιουργηθούν τα κατάλληλα **containers** πρέπει να εκτελεστεί η παρακάτω εντολή **docker-compose up --build**

Είναι σημαντικό ο χρήστης να βρίσκεται στο **root directory** δηλαδή στο *Ebay_clone* ώστε να βρίσκεται στο ίδιο **directory** με το **docker-compose.yml**. Το **docker compose** δημιουργεί δύο **services**:

1. Backend

Αυτό είναι το **container** για το **backend** μέρος της εργασίας μας. Εδώ ορίζεται το **port** από το οποίο θα λαμβάνει αιτήματα και ο αποθηκευτικός χώρος που του έχει ανατεθεί. Επίσης έχουν δοθεί ένα **flag** το οποίο φροντίζει να επανεκινείται το **container** σε περίπτωση που υπάρξει κάποιο σφάλμα. Έτσι εξασφαλίζεται ότι υπάρχει χαμηλό **downtime** καθώς το σύστημα δεν καταρέει με το πρώτο σφάλμα.

2. Frontend

ομοίως με το **frontend** εδώ ορίζεται το **port** και ο αποθηκευτικός χώρος καθώς και το **flag** για την επανεκίνηση. Επίσης δηλώνεται ότι το **frontend** εξαρτάται από το **backend** οπότε για να ξεκινήσει την λειτουργία του θα πρέπει πρώτα να έχει εκκινήσει το **backend**

Για κάθε ένα από αυτά τα **services** υπάρχει ένα αντίστοιχο **dockerfile**

1. Dockerfile-backend

Αυτό το **dockerfile** είναι βασισμένο σε ένα **python image** και κατεβάζει όλα τα **dependencies** που είναι απαραίτητα για την εκτέλεση του προγράμματος μέσα από το αρχείο **requirements.txt**.

Στην συνέχεια εκτελείται ένα **bash script** το οποίο είναι υπεύθυνο για την αρχικοποίηση των δεδομένων και την εκκίνηση του **server**.

Μέσα στο **script** αυτό δημιουργούνται τα **certificates** για την λειτουργία του **ssl/tls** ενώ δίνεται και η δυνατότητα της εξολοκλήρου αναδημιουργίας της βάσης δεδομένων σαν μια μορφή **backup recovery**. Αυτό ελεγχεται μέσω των **environment variables** **RESET_DB** και **LOAD_USERS**.

2. Dockerfile-frontend

Αυτό το **dockerfile** είναι βασισμένο σε ένα **node image** και ευθύνεται για την ύπαρξη των απαραίτητων **dependencies** πράγμα που επιτυγχάνεται μέσω του **npm install**. Στην συνέχεια όπως ακριβώς και στο **backend** εκτελείτε ένα **bash script**. Αυτή την φορά όμως αυτό απλα δημιουργεί τα **certificates** και ξεκινάει την εκτέλεση του **node**

Για τον τερματισμό της εκτέλεση του προγράμματος μπορεί να χρησιμοποιηθεί το παρακάτω **command**:

```
docker compose down
```

3. Backend Django

3.1. Εισαγωγή

Για την υλοποίηση του νοτιαίου άκρου της εφαρμογής χρησιμοποιήθηκε η βιβλιοθήκη **Django** της **python** λόγω της υποστήριξης της για **orm** μοντέλα καθώς και λόγω την ευκολίας χρήσης της. Μία εφαρμογή **django** είναι οργανωμένη σε **apps** το καθένα από τα οποία εκτελεί κάποιες λειτουργίες. Το κάθε **app** δεν είναι απαραίτητο να είναι ανεξάρτητο από τα άλλα, αλλά έχει καλά ορισμένη λειτουργία και τα **apps** ομαδοποιούνται με βάση αυτήν. Παρακάτω βρίσκεται μία σύντομη αναφορά στα **apps** που χρησιμοποιήθηκαν στην εφαρμογή μας καθώς και μία επεξήγηση της λειτουργικότητάς τους.

3.2. app summary

1. Γενική δομή Κάθε **app** αποτελείται από κάποια **components** τα οποία είναι ίδιας μορφής ώστε να υπάρχει οργάνωση και να είναι εύκολη η κατανόηση της λειτουργίας του κώδικα. Τα **components** αυτά αναλύονται παρακάτω
 - (a) **models** Ορίζουν τη δομή των δεδομένων και τη σχέση τους με τη βάση δεδομένων.
 - (b) **serialisers** Μετατρέπουν τα δεδομένα των **models** σε μορφή **JSON** για την ανταλλαγή πληροφοριών μέσω **API**, και αντίστροφα.
 - (c) **urls** Ορίζουν τα **endpoints** του **app** στα οποία θα έχει πρόσβαση το **frontend**
 - (d) **views** Περιέχουν τον κώδικα και τις συναρτήσεις που θα εκτελεστούν κάθε φορά που υπάρχει κάποιο αίτημα σε κάποιο **url**

2. backend

Σε κάθε **django project** υπάρχει ένα ειδικό **app** το οποίο ορίζει παραμέτρους σχετικά με τον τρόπο που θα εκτελεστεί το πρόγραμμα. Αυτό περιέχει:

(a) wsgi-asgi

Το **wsgi** και το **asgi** καθορίζουν τον τρόπο με τον οποίο γίνεται η διαχείριση των **requests** από το **frontend**. Το **wsgi** χειρίζεται τα αιτήματα ένα ένα ενώ το **asgi** επιτέπει την ταυτόχρονη διαχείριση τους

(b) settings

Οι ρυθμίσεις περιέχουν όλα τα **apps** που θα χρησιμοποιηθούν, όχι μόνο αυτά που έχουν δημιουργηθεί από τον χρήστη αλλά και τα ήδη υπάρχοντα και οι διάφορες επιλογές που έχουν οριστεί καθορίζουν τον τρόπο χρήσης τους

(c) urls

Τα **urls** δεν ορίζουν κάποιο συγκεκριμένο **endpoint** αλλά κατευθύνουν τα εισερχόμενα αιτήματα στα **urls** που ορίζονται μέσα στα υπόλοιπα **apps**

3. auctions Εδώ αποθηκεύονται όλες οι πληροφορίες σχετικά με τις δημοπρασίες και τα αντικείμενα που πωλούνται. Τα μοντέλα είναι:

(a) Item

Περιέχει όλες τις άμεσες πληροφορίες σχετικά με τα αντικείμενα

(b) Category

Περιέχει συγκεκριμένες κατηγορίες που δημιουργούνται κατά την εκκίνηση. Δεν έχουμε δώσει στον χρήστη την δυνατότητα να δημιουργεί δικές του κατηγορίες για να αποφύγουμε την υπαρξη διαφορετικών κατηγοριων πχ "Μπλουζα", "μπλουζες" που θα δυσκόλευαν τον χρήστη να βρει αυτό που θέλει

(c) Bid

Κρατάει ποιος έχει κάνει **bid** σε ποιο αντικείμενο

(d) ItemImage

Εδώ βρίσκονται οι εικόνες που πιθανώς να υπάρχουν για κάποιο αντικείμενο

Στο αρχείο **views.py** βρίσκονται διάφορες συναρτήσεις που επιτρέπουν στον χρήστη να λάβει δεδομένα από την βάση καθώς και να τροποποιήσει τα δεδομένα της στις περιπτώσεις που αυτό χρειάζεται. Έχουν ληφθεί τα απαραίτητα μέτρα, με την χρήση **custom permission classes**, ώστε ο κάθε χρήστης να έχει πρόσβαση μόνο στα δεδομένα που θα έπρεπε να του είναι διαθέσιμα

4. config Σε αυτό το app υλοποιείται ένα custom command το οποίο μπορεί να χρησιμοποιήσει ένα διαχειριστής του server με το οποίο μπορούν να φορτωθούν οι χρήστες που είναι αποθηκευμένοι στην βάση. Αυτό είναι το command που χρησιμοποιήθηκε κατά την έναρξη του προγράμματος για την αρχική εισαγωγή στοιχείων στην βάση

5. user

Σε αυτό το **app** βρίσκονται όλοι οι χρήστες που έχουν λογαριασμό στην ιστοσελίδα. Επίσης εδώ είναι και ο λογαριασμός του διαχειριστή ο οποίος διαφέρει από τους υπόλοιπους μόνο στο ότι έχει κάποια παραπάνω **permissions**. Τα **permissions** αυτά ορίζονται στο **permissions.py**. Εκεί ορίζεται και ένα ακόμη **permission**, αυτό που χρησιμοποιείται για να εξεταστεί αν κάποιος χρήστης έχει εγκριθεί ή όχι από τον διαχειριστή

6. *user_messages*

Τα *user_messages* χειρίζονται όλη την λειτουργικότητα σχετικά με τα μηνύματα μεταξύ χρηστών. Υπάρχουν 2 **models**.

(a) **Conversations**

Περιέχει τις συζητήσεις μεταξύ χρηστών

(b) **Messages**

Κάθε **message** ανήκει σε μία συζήτηση. Αυτό το μήνυμα είναι δηλωμένο ως μη αναγνωσμένο μέχρι να το αναζητήσει ένας χρήστης μέσα από μία συζήτηση

Κάθε χρήστης που έχει λογαριασμό μπορεί να διαβάσει τα εισερχόμενα καθώς και τα εξερχόμενα μηνύματα του ή να διαγράψει κάποιο υπάρχον μήνυμα. Στην περίπτωση της διαγραφής αρχικά δηλώνεται ως διαγραμμένο στο **backend** έχουμε δηλαδή **soft delete** και το μήνυμα αυτό δεν εμφανίζεται μόνο για τον χρήστη που το διέγραψε. Ενώ στην περίπτωση που το μήνυμα διαγραφεί και από τους δύο χρήστες τότε γίνεται οριστική διαγραφή

7. **authentication**

Για να γνωρίζουμε ποιοι χρήστες έχουν εισέλθει στον ιστότοπο σωστά και έχουν τα απαραίτητα **credentials** χρησιμοποιείται **JWT**. Το **django** παρέχει από μόνο του υποβιβλιοθήκη η οποία προσφέρει την λειτουργία αυτή. Αυτή μας δίνει την δυνατότητα να καλούμε ένα έτοιμο **view** που χειρίζεται το **authentication** για την αρχική είσοδο του χρήστη. Μετά απλά αποθηκεύουμε το **token** στο **frontend** και το χρησιμοποιούμε στην συνέχεια.

Χρειάστηκε όμως να προσθέσουμε κάποιες επιπλέον πληροφορίες όπως αν ο χρήστης είναι διαχειριστής ή αν έχει εγκριθεί. Αυτό το κάναμε μέσω μιας υποκλάσης του έτοιμου **view**

8. **recomendation system**

Το **recomendation system** υλοποιήθηκε με βάση τον αλγόριθμο **matrix factorisation**. Αυτό το σύστημα λαμβάνει τα **items** που θα λάμβανε ο χρήστης με τυχαία σειρά και δημιουργεί την καλύτερη δυνατή ταξινόμηση τους. Αυτό γίνεται με βάση 2 μοντέλα που είναι αποθηκευμένα στην βάση. Το ένα χρησιμοποιείτε για προτάσεις όταν ο χρήστης έχει κάνει **bids** και το άλλο όταν έχει πιστευτεί κάποια **items**. Στην περίπτωση που δεν υπάρχει τίποτε από τα δύο η πρόταση αντικειμένων γίνεται τυχαία.

Τα μοντέλα αυτά έχουν σωστές προτάσεις για μια συγκεκριμένη χρονική στιγμή, οπότε θα πρέπει να επανεκπαιδεύεται για να αντιπροσωπεύει την πραγματικότητα. Αποφασίσαμε να γίνεται αυτό όταν συμβαίνει ένας συγκεκριμένος αριθμός από συμβάντα, δηλαδή να έχουν γίνει **bids** και επισκέψεις σε αντικείμενα. Τώρα αυτός ο αριθμός είναι 1 για να διευκολυνθεί η διαδικασία ελέγχου αλλά σε περιβάλλον **deployment** θα ήταν αρκετά παραπάνω καθώς διαφορετικά θα υπερφορτωνόταν ο διακομιστής

9. **SSL-TLS** Για την κρυπτογράφηση των αιτημάτων χρησιμοποιήθηκαν **self signed certificates** καθώς δεν ήταν δυνατό να αποκτήσουμε κανονικά. Αυτό έχει ως αποτέλεσμα τα **browser** να το θεωρούν κενό ασφαλείας και να προηδοποιούν τον χρήστη. Σε περιβάλλον **deployment** θα υπήρχαν **CA signed certificates** αντί για αυτά που έχουμε τώρα. Τα **certificates** αυτά δημιουργούνται στα **startup scripts** και χρησιμοποιούνται καθ' όλη την διάρκεια εκτέλεσης του προγράμματος.

Για την χρήση τους υπάρχουν οι κατάλληλες ρυθμίσεις στο **django** (**django-sslserver**) και στο **vite** (ρυθμίσεις **https**)

4. Frontend React

4.1. Εισαγωγή

Το Frontend έχει υλοποιηθεί πλήρως με Typescript με React και Tailwind CSS . Τα κύρια χαρακτηριστικά είναι:

1. Reactive Pages

Τα Pages αποτελούν την κύρια εμφάνιση της εργασίας (π.χ. εγγραφή, δημιουργία Δημοπορασίας, μηνύματα). Αυτές οι σελίδες ή φόρμες είναι η ίδια η διεπαφή ανάμεσα στο Frontend και στο Backend.

2. Routes

Τα Routes (π.χ. /admin, /ebay, /login) αποτελούν το τρόπο μεταφοράς ανάμεσα στα Pages. Ακόμα τα Routes είναι προστατευμένα ώστε να μην έχουν πρόσβαση όσοι χρήστες δεν έχουν εξουσιοδοτηθεί. Πρέπει να σημειωθεί πως για οποιοδήποτε λάθος Route η εφαρμογή θα κατεύθυνει τον user στην σελίδα Login.

3. Components

Τα Components αποτελούν τις ενέργειες που μπορεί να κάνει ένας χρήστης. Έχουν δημιουργηθεί με τέτοιο τρόπο, ώστε να είναι εύκολονόητο τι μπορείς ή δεν μπορείς να κάνεις μόνο με μία ματιά. Αυτά περιλαμβάνουν καρτες, κουμπια και alerts.

4.2. Pages summaray

(a) Admin-AdminPage

Είναι η σελίδα διαχείρισης εγγεγραμμένων χρωστων στην οποία ο διαχειριστής μπορεί να ελέγχει τα στοιχεία των χρηστων και να διαγράφει χρήστες τους οποίους βρήσκει να μην ακολουθούνε τους κανόνες.

(b) Admin-RequestPage

Αντίστοιχα είναι η σελίδα διαχείρισης μη-εγγεγραμμένων χρωστων στην οποία ο διαχειριστής μπορεί να ελέγχει τα στοιχεία των χρηστων και μετα να αποφασίζει αν θα αποδεχτεί ή θα απορρίψει την αίτηση εγγραφής τους.

(c) Login-Login

Αυτή είναι η πρώτη σελίδα που θα βλέπουν όλοι όσοι ανοίγουν την εφαρμογή. Εδω εισάγοντας σωστά το Username και το Password ένας χρήστης θα εισέρχεται στην εφαρμογή. Ως διαχειρηστής υπάρχει ήδη ένας χρήστης

(d) Login-register

Αυτή η σελίδα χρειάζεται για να κάνουν οι νέοι χρήστες αίτηση εγγραφής. Προβάλλονται σημειώσεις για την ομαλή εμπειρία εγγραφής και alerts αν δεν έχει συμπληρωθεί σωστά η φόρμα.

(e) Login-WatingPage

Στην σωστή υποβολή αίτησης ο νέος χρήστης οδηγείται σε αυτήν την σελίδα μέχρι να γίνει αποδεκτό το αίτημα του στην εφαρμογή.

(f) Login-homepage

Στην σωστή πρόσβαση ο χειριστής(είτε ο χειριστής) έρχεται σε αυτήν την σελίδα και επιλέγει το τι θέλει να κάνει.

(g) Auction-EbayPage

Είναι η κεντρική σελίδα αναζήτησης / πλοήγησης δημοπρασιών στην οποία εγγεγραμμένοι και μη-εγγεγραμμένοι χρήστες μπορούν να αναζητήσουν δημοπρασίες με βάση το όνομα, την τοποθεσία, και με φίλτρα.

(h) Auction-myAuction

Είναι η κύρια σελίδα διαχείρισης των δημοπρασιών ενός χρήστη. Εδώ μπορεί να βλέπει ο κάθε εγγεγραμμένος χρήστης της δημοπρασίες του και να της επεξεργάζεται αν δεν έχει δωθεί ακόμα η πρώτη προσφορά.

(i) Auction-makeAuction

Αυτή η σελίδα αποτελεί την φόρμα δημιουργίας μια νέας δημοπρασίας.

(j) Auction-mybids

Σε αυτήν την σελίδα ο χρήστης μπορεί να ελέγχει τις προσφορές που έχει δηλώσει στις διάφορες δημοπρασίες αλλά και την καλύτερη προσφορά που έχει γίνει στην δημοπρασία.

(k) Auction-ItemBids

Σε αυτήν την σελίδα ο χρήστης μπορεί να ελέγχει της προσφορες που έχουν δηλώσει άλλοι χρήστες για την κάθε δημοπρασία του. Όπου και η καλύτερη προσφορά θα νικήσει.

(l) Chat-ChatIn/ChatOut

Αυτές οι δύο σελίδες είναι παρόμοιες. Αποτελούν τον τρόπο με τον οποίο όλοι οι χρήστες επικοινωνούν μεταξύ τους, αμέσως μετα απο μια λήξη μιας δημοπρασίας. Ακόμα οι χρήστες, αν θέλουν μπορούν να βαθμολογήσουν τον άλλο χρήστη αρνητικά ή θετικά.

4.3. Τεστ

Εδώ έχουμε μερικούς δοκιμαστικούς λογαριασμών για οποιοδήποτε τεστ χρειαζόμασταν. Στην περίπτωση που χρειαζόμασταν κιάλους χρήστες μπορούσαμε και πολύ εύκολα να τους δημιουργήσουμε.

Username	Password	User	Amdin
Admin	test1234	Ναι	Ναι
Test	test1234	Ναι	Οχι
BookStore	test1234	Ναι	Οχι

Table 1: Trials

5. Epilogue

5.1. Problems

1. Ένα κύριο πρόβλημα ήταν η σωστή σειρά μηνυμάτων και συνομιλιών μεταξύ τους διαφορετικούς χρήστες. Ακόμα χρειαζόμασταν ενδηξεις για τα μη-αναγνωσμένα μηνήματα.

5.2. Summary

Η εφαρμογή αυτή υλοποιήθηκε ως ένα ολοκληρωμένο περιβάλλον που συνδυάζει λειτουργικότητα και ασφάλεια. Υποστηρίζει πολλαπλούς ρόλους χρηστών, παρέχει δυνατότητες δημιουργίας και διαχείρισης δημοπρασιών, προσφέρει σύστημα ανταλλαγής μηνυμάτων και περιλαμβάνει μηχανισμό συστάσεων. Η αξιοποίηση τεχνολογιών όπως Django, React/TypeScript, TailwindCSS και Docker, σε συνδυασμό με JWT authentication και SSL/TLS, συνέβαλε στη δημιουργία ενός σύγχρονου και ασφαλούς οικοσυστήματος.